

PYTHON FOR LOOPS

PYTHON FOR LOOPS

- A **for** loop is used for iterating over a sequence (that is either a list, a tuple, a dictionary, a set, or a string).
- This is less like the **for** keyword in other programming languages, and works more like an **iterator** method as found in other object-orientated programming languages.
- With the **for** loop we can execute a set of statements, once for each item in a **list, tuple, set** etc.

WHAT IS FOR LOOP IN PYTHON?

Syntax of for Loop

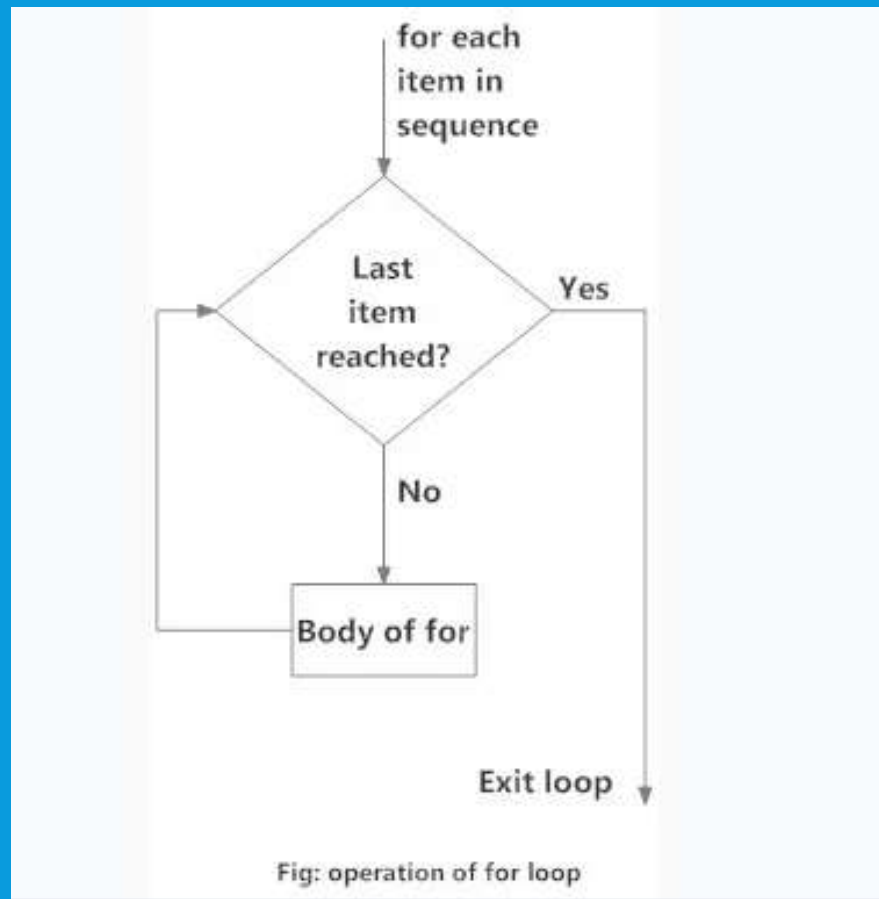
`for val in sequence:`

`loop body`

- **val** is the **variable** that takes the value of the item inside the sequence on each iteration.
- **Loop** continues until we reach the **last item** in the sequence.
- The body of for loop is separated from the rest of the code using indentation.

WHAT IS FOR LOOP IN PYTHON?

Flowchart of for Loop



EXAMPLE

Print each fruit in a fruit list:

```
fruits = ["apple", "banana", "cherry"]  
for x in fruits:  
    print(x)
```

OUTPUT

```
apple  
banana  
cherry
```

LOOPING THROUGH A STRING

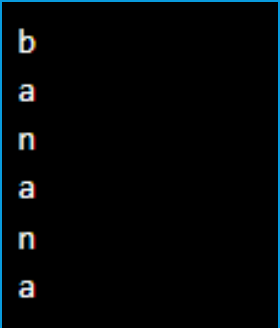
- Even strings are iterable objects, they contain a sequence of characters:

Example

- Loop through the letters in the word "banana":

```
for x in "banana":  
    print(x)
```

OUTPUT



```
b  
a  
n  
a  
n  
a
```

THE **BREAK** STATEMENT

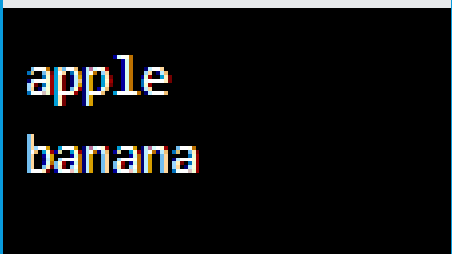
- With the **break** statement we can stop the loop before it has looped through all the items:

Example

Exit the loop when x is "banana":

```
fruits = ["apple", "banana", "cherry"]  
for x in fruits:  
    print(x)  
    if x == "banana":  
        break
```

OUTPUT:



```
apple  
banana
```

SAMPLE PROBLEM

Exit the loop when `x` is "banana" and print the value of `x`:

THE **CONTINUE** STATEMENT

- With the **continue** statement we can stop the current iteration of the loop, and continue with the next:

Example

Do not print banana:

```
fruits = ["apple", "banana", "cherry"]  
for x in fruits:  
    if x == "banana":  
        continue  
    print(x)
```

OUTPUT

```
apple  
cherry
```

THE **RANGE()** FUNCTION

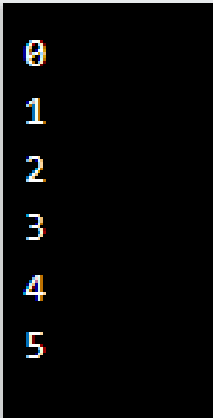
- To loop through a set of code a specified number of times, we can use the **range()** function,
- The **range()** function returns a sequence of numbers, starting from 0 by default, and increments by 1 (by default), and ends at a specified number.

Example

Using the range() function:

```
for x in range(6):  
    print(x)
```

OUTPUT:



```
0  
1  
2  
3  
4  
5
```

Note that **range(6)** is not the values of 0 to 6, but the values 0 to 5.

THE **RANGE()** FUNCTION

- The `range()` function defaults to 0 as a starting value, however it is possible to specify the starting value by adding a parameter: `range(2, 6)`, which means values from 2 to 6 (but not including 6):

Example

- Using the start parameter:

```
for x in range(2, 6):  
    print(x)
```

OUTPUT:

```
2  
3  
4  
5
```

THE **RANGE()** FUNCTION

- The `range()` function defaults to increment the sequence by 1, however it is possible to specify the increment value by adding a third parameter: `range(2, 30, 3)`:

Example

Increment the sequence with 3 (default is 1):

```
for x in range(2, 30, 3):  
    print(x)
```

OUTPUT

```
2  
5  
8  
11  
14  
17  
20  
23  
26  
29
```

ELSE IN FOR LOOP

- The **else** keyword in a for loop specifies a block of code to be executed when the loop is finished:

Example

Print all numbers from 0 to 5, and print a message when the loop has ended:

```
for x in range(6):  
    print(x)  
else:  
    print("Finally finished!")
```

Note: The **else** block will **NOT** be executed if the loop is stopped by a **break** statement.

OUTPUT

```
0  
1  
2  
3  
4  
5  
Finally finished!
```

ELSE IN FOR LOOP

Example

- Break the loop when **x** is 3, and see what happens with the **else** block:

```
for x in range(6):  
    if x == 3: break  
    print(x)  
else:  
    print("Finally finished!")
```

#If the loop breaks, the else block is not executed.

OUTPUT

0

1

2

NESTED LOOPS

- A nested **loop** is a **loop** inside a **loop**.
- The "**inner loop**" will be executed one time for each iteration of the "**outer loop**":

Example

Print each adjective for every fruit:

```
adj = ["red", "big", "tasty"]  
fruits = ["apple", "banana", "cherry"]
```

```
for x in adj:  
    for y in fruits:  
        print(x, y)
```

OUTPUT

```
red apple  
red banana  
red cherry  
big apple  
big banana  
big cherry  
tasty apple  
tasty banana  
tasty cherry
```

THE **PASS** STATEMENT

- **for** loops cannot be empty, but if you for some reason have a **for** loop with no content, put in the **pass** statement to avoid getting an error.

Example

```
for x in [0, 1, 2]:  
    pass
```

having an empty for loop like this, would raise an error without the pass statement

THANK YOU